Sprint 2 — Spatial & indexed queries over 64k European train stations

In this sprint you'll work with a real-world dataset of ~64000 European railway stations (station, latitude/longitude, country, time_zone, time_zone_group, and flags such as *is_city*, *is_main_station*, *is_airport*). The goal is to build efficient, deterministic query engines on top of this data using the **BST/AVL** implementations already made in classes and making a from-scratch **2D-tree** for geographic queries.

USEI06 — Time-Zone Index and Windowed Queries

Context
Planners need fast, reproducible lookups of stations by **time zone group**, and convenient searches by **station name** and **country** without scanning the 64k-row CSV.

User Story
As a planner, I want a set of **BST/AVL trees** that lets me quickly search by:

- latitude
- longitude
- time zone group and country
  So I can efficiently retrieve **all stations of a time zone group** by ascending order of country or in a **window of time zone groups**, such as ['CET', 'WET/GMT'].

Acceptance Criteria
Validate: non-empty name station, country, and time zone group; latitude ∈ [−90,90], longitude ∈ [−180,180]. Reject invalid rows.
If multiple stations share the same coordinates, such as Lisbon Santa Apolónia and Lisbon Oriente, the tree must preserve all these stations sorted by name in ascending order.

Returns
3–5 sample queries.
Temporal analysis complexity.

USEI07 — Build a balanced 2D-Tree on Latitude/Longitude

Context
We need a spatial index over ~64k European train stations to enable fast geographic queries. A **KD-tree (k=2)** on (latitude, longitude) supports range and nearest-neighbour operations much faster than scanning all stations.

User Story
As a data engineer, I want to **build a balanced 2D-tree** using (lat, lon) so that range and proximity searches run efficiently and deterministically.

Acceptance Criteria
The simplest way to ensure a balance 2D-tree is to perform a bulk build; for this, develop a **balanced build strategy** that will use the BST/AVL of latitude and longitude created in USEI01 to efficiently construct the 2D-tree. If multiple stations share the same coordinates, such as

38.71387, -9.122271, which are linked to Lisbon Santa Apolónia and Lisbon Oriente, the 2D-tree must preserve all these stations sorted by name in ascending order.

Returns
Size of the tree, its height, and all distinct bucket sizes.
Temporal analysis complexity.

US08 — Search by Geographical Area

Context
Operations need to list **all stations inside a map rectangle**, optionally filtered by type or country. KD-tree pruning should avoid scanning the whole dataset.

User Story
As a planner, I want to query the 2D-tree for all stations in a **latitude/longitude range** with optional filters, so I can extract relevant stations quickly.

Acceptance Criteria
The region: lat ∈ [latMin, latMax], lon ∈ [lonMin, lonMax] (inclusive).

The search should allow the optional application of filters with the following criteria:
- is_city (true | false)
- is_main_station (true | false)
- country (PT | ES | all)

Returns
3–5 sample queries.
Temporal analysis complexity.

USEI09 — Proximity Search (Nearest-N with Filters)

Context
Users need the **N closest stations** to a target coordinate (e.g., for nearest major hubs). 2D-tree should minimize explored nodes.

User Story
As an analyst, I want to find the **nearest N stations** to a given (lat, lon) using the 2D-tree, with optional filters, so I can get accurate nearby results efficiently.

Acceptance Criteria
Use Haversine distance (km) with Earth radius between two (lat,lon) points[1].
The search should enable optional filtering based on the time zone criteria.

Returns
A list with n nearest neighbours.
Temporal analysis complexity.

[1] http://www.movable-type.co.uk/scripts/latlong.html

USEI10 — Radius search and density summary

Context
Operational questions often ask, “what stations lie within **R km** of a point?” and “how are they distributed by country and is_city?” We need both the list and a quick summary.

User Story
As an operations planner, I want to fetch all stations within a **radius R (km)** of a target (lat, lon) using the 2D-tree, and get a summary by country and by is_city, so I can assess local coverage.

Acceptance Criteria
Use Haversine distance (km) with Earth radius between two (lat,lon) points.

Returns
A BST/AVL tree sorted by distance (ASC), and station name (DESC).
Temporal analysis complexity.

Quotations

USEI06 – 15%
USEI07 – 30%
USEI08 – 15%
USEI09 – 20%
USEI10 – 20%

For each US:
- Code (60%)
- Test (25%)
- Complexity Analysis (15%)